

RELAZIONE TECNICA – PROGETTO M2M (Modern Management – Ristorante)

1. Introduzione

Il progetto **M2M – Modern Management (Ristorante)** è una piattaforma digitale full-stack progettata per la gestione moderna di un ristorante.

Il sistema simula un ambiente reale in cui diverse aree operative (sala, cucina e amministrazione) comunicano tra loro in tempo reale attraverso un'infrastruttura centralizzata e scalabile.

L'obiettivo principale è realizzare un sistema informatico in grado di digitalizzare e ottimizzare la gestione degli ordini e delle informazioni interne di un ristorante, riducendo errori e tempi di comunicazione.

Il progetto è sviluppato utilizzando tecnologie moderne come **Next.js, React, TypeScript e Supabase**, con supporto backend basato su **Node.js**.

2. Descrizione del progetto

Il sistema M2M è strutturato come una piattaforma multi-modulo.

Ogni modulo rappresenta un'area del ristorante e può:

- inviare dati (es. nuovi ordini dalla sala)
- ricevere aggiornamenti (es. stato preparazione cucina)
- sincronizzarsi in tempo reale con gli altri moduli

Il sistema permette quindi di simulare un flusso operativo realistico, in cui tutte le informazioni sono centralizzate e aggiornate istantaneamente.

3. Obiettivo del progetto

Gli obiettivi principali del progetto sono:

- digitalizzare la gestione degli ordini
- simulare la comunicazione sala ↔ cucina
- implementare aggiornamenti in tempo reale
- utilizzare architetture moderne full-stack
- applicare concetti di API e backend distribuito

Dal punto di vista didattico, il progetto consente di applicare tecnologie reali utilizzate nello sviluppo software professionale.

4. Problema che risolve

Nei ristoranti tradizionali, la comunicazione tra sala e cucina è spesso:

- lenta
- soggetta a errori umani
- non centralizzata

Questo può causare ritardi, confusione e perdita di informazioni.

Il sistema M2M risolve questi problemi grazie a:

- comunicazione digitale immediata
- database centralizzato
- aggiornamenti in tempo reale
- sincronizzazione automatica dei dati

5. Architettura del sistema

Il progetto segue un'architettura **client-server moderna**.

◇ Frontend (Next.js + React)

Gestisce l'interfaccia utente e la visualizzazione dei dati.

◇ Backend (Node.js + API)

Gestisce la logica applicativa e le comunicazioni con il database.

◇ Database e Backend as a Service (Supabase)

Fornisce:

- database PostgreSQL
- autenticazione utenti
- realtime updates

◇ Comunicazione API

- REST API per operazioni standard
- WebSocket (Realtime) per aggiornamenti istantanei

6. TECNOLOGIE E LIBRERIE UTILIZZATE (APPROFONDITE)

Next.js

Descrizione

Next.js è un framework React full-stack che permette lo sviluppo di applicazioni web moderne con rendering lato server (SSR), routing automatico e API integrate.

Utilizzo nel progetto

Nel progetto M2M viene utilizzato per:

- creare pagine dell'applicazione
- gestire routing automatico
- integrare API backend
- migliorare performance tramite SSR

React

Descrizione

React è una libreria JavaScript per la costruzione di interfacce utente basate su componenti riutilizzabili.

Utilizzo nel progetto

Serve per:

- creare componenti (ordini, tavoli, cucina)
- aggiornare dinamicamente l'interfaccia
- gestire lo stato dell'applicazione

TypeScript

Descrizione

TypeScript è un superset di JavaScript che introduce tipizzazione statica.

Utilizzo nel progetto

Nel progetto viene utilizzato per:

- ridurre errori di codice
- migliorare la manutenzione
- definire modelli dati (ordini, utenti, tavoli)

Tailwind CSS

Descrizione

Tailwind CSS è un framework CSS utility-first che permette di creare interfacce rapidamente.

Utilizzo nel progetto

Serve per:

- styling dell'interfaccia
- layout responsive
- design coerente tra componenti

React Context API

Descrizione

La Context API è un sistema di gestione dello stato globale in React.

Utilizzo nel progetto

Utilizzata per:

- condividere dati tra moduli (sala, cucina, admin)
- evitare prop drilling
- gestire stato globale degli ordini

Node.js

Descrizione

Node.js è un runtime JavaScript lato server basato sul motore V8 di Chrome.

Utilizzo nel progetto

Serve per:

- gestire logica backend
- creare API personalizzate
- comunicare con Supabase
- gestire richieste HTTP

Supabase

Descrizione

Supabase è una piattaforma Backend-as-a-Service basata su PostgreSQL.

Fornisce:

- database cloud
- autenticazione utenti
- API automatiche
- realtime WebSocket

Utilizzo nel progetto

- salvataggio ordini
- gestione utenti
- sincronizzazione dati
- aggiornamenti in tempo reale

Supabase Auth

Descrizione

Sistema di autenticazione integrato di Supabase.

Utilizzo nel progetto

- login utenti
- gestione sessioni
- controllo accessi (ruoli sala/cucina/admin)

Supabase Realtime (WebSocket)

Descrizione

Sistema di comunicazione in tempo reale basato su WebSocket.

Utilizzo nel progetto

- aggiornamento ordini istantaneo
- sincronizzazione cucina ↔ sala
- notifiche automatiche

REST API

Descrizione

Le REST API sono un'architettura che permette la comunicazione tra client e server tramite HTTP.

Metodi principali:

- GET → leggere dati
- POST → creare dati
- PUT → aggiornare dati
- DELETE → eliminare dati

Utilizzo nel progetto

- invio ordini
- aggiornamento stato ordini
- recupero dati dal database

dotenv (.env)

Descrizione

Libreria che gestisce variabili d'ambiente in modo sicuro.

Utilizzo nel progetto

- protezione API key
- configurazione Supabase
- separazione dati sensibili dal codice

Configurazione

SUPABASE_URL=...

SUPABASE_ANON_KEY=...

npm (Node Package Manager)

Descrizione

Gestore ufficiale dei pacchetti Node.js.

Utilizzo nel progetto

- installazione librerie
- gestione dipendenze
- esecuzione script

Comandi

`npm install`

`npm run dev`

Git & GitHub

Descrizione

- Git: sistema di versionamento del codice
- GitHub: piattaforma cloud per repository

Utilizzo nel progetto

- controllo versioni
- backup del codice
- collaborazione
- pubblicazione progetto

7. API UTILIZZATE

Supabase API

- generate automaticamente
- gestione database
- operazioni CRUD
- realtime integration

API Backend (Node.js)

- logica personalizzata
- elaborazione dati
- comunicazione con Supabase

WebSocket (Realtime API)

- aggiornamenti istantanei
- sincronizzazione moduli
- eliminazione polling

8. CONFIGURAZIONE DEL PROGETTO

Il progetto richiede un file:

`.env.local`

Contenente:

- URL Supabase
- API key
- configurazione autenticazione
- attivazione realtime

9. AVVIO DEL PROGETTO

```
npm install
```

```
npm run dev
```

Il sistema avvia:

- frontend Next.js
- backend Node.js
- connessione Supabase

10. FUNZIONAMENTO DEL SISTEMA

Il flusso operativo è il seguente:

1. L'ordine viene creato dalla sala
2. Il frontend invia i dati al backend
3. Node.js elabora e salva su Supabase
4. Supabase aggiorna il database
5. Realtime notifica la cucina
6. La cucina riceve l'aggiornamento
7. Lo stato dell'ordine viene aggiornato in tempo reale

11. CONCLUSIONE

Il progetto **M2M – Modern Management (Ristorante)** rappresenta una simulazione realistica di un sistema di gestione ristorativa digitale.

Grazie all'utilizzo di tecnologie moderne come **Next.js, React, TypeScript e Supabase**, è stato possibile sviluppare una piattaforma scalabile, reattiva e in tempo reale.

Il progetto ha una forte valenza didattica in quanto permette di comprendere:

- architetture full-stack moderne

- comunicazione client-server
- API REST e WebSocket
- sistemi realtime distribuiti

12. Struttura logica del database (Supabase)

Il database del progetto M2M è realizzato tramite **Supabase (PostgreSQL)** ed è progettato secondo una struttura relazionale semplice ma efficace, in grado di supportare tutte le funzionalità del sistema di gestione ristorativa.

L'obiettivo del database è centralizzare le informazioni e garantire la sincronizzazione in tempo reale tra i diversi moduli del sistema (sala, cucina e amministrazione).

Le principali entità del sistema sono rappresentate da quattro tabelle fondamentali.

Tabella orders (Ordini)

Questa tabella rappresenta il cuore del sistema, poiché gestisce tutto il ciclo di vita degli ordini.

Funzione

Memorizza gli ordini creati dalla sala e ne traccia l'evoluzione fino al completamento.

Struttura logica

Ogni ordine contiene:

- identificativo univoco (UUID)
- numero del tavolo associato
- elenco dei prodotti ordinati
- stato dell'ordine
- timestamp di creazione

Logica di funzionamento

Quando un ordine viene creato:

1. viene inserito nel database
2. Supabase lo sincronizza in realtime
3. la cucina riceve immediatamente l'aggiornamento
4. lo stato dell'ordine viene aggiornato durante la preparazione

Tabella users (Utenti)

Funzione

Gestisce gli utenti autorizzati ad accedere al sistema.

Struttura logica

Ogni utente è definito da:

- ID univoco
- email
- ruolo (sala, cucina, amministratore)

Utilizzo

Questa tabella consente l'implementazione del controllo degli accessi, garantendo che ogni utente possa accedere solo alle funzionalità del proprio ruolo.

Tabella menu_items (Menu)

Funzione

Contiene tutti i prodotti disponibili nel menu del ristorante.

Struttura logica

Ogni elemento del menu è composto da:

- nome del piatto
- prezzo
- categoria (es. primi, secondi, bevande)

Utilizzo

Viene utilizzata dalla sala per la creazione degli ordini, garantendo coerenza dei dati inseriti.

Tabella tables (Tavoli)

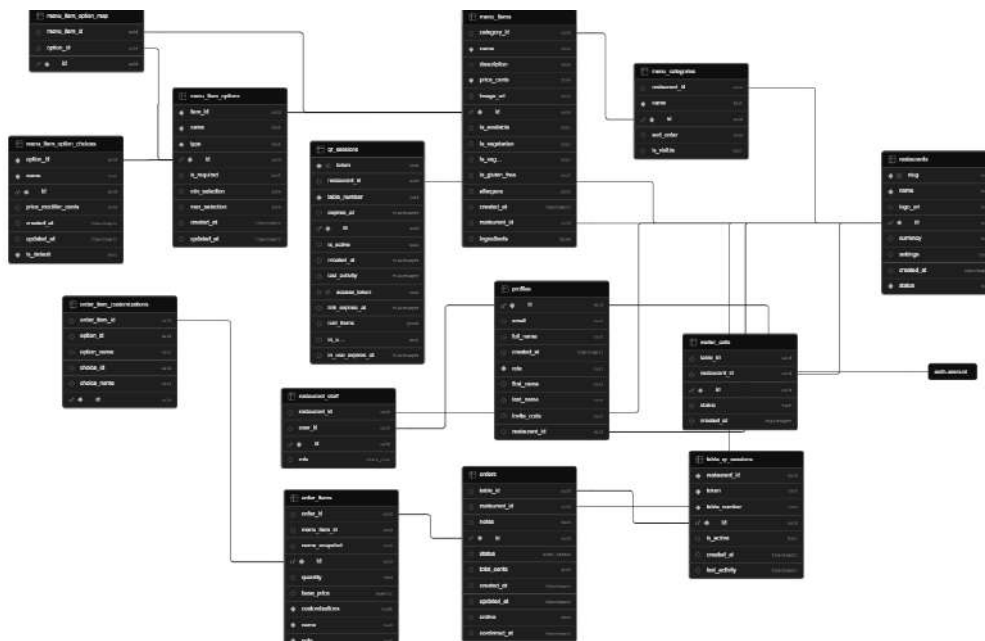
Funzione

Gestisce lo stato dei tavoli presenti nel ristorante.

Struttura logica

Ogni tavolo contiene:

- numero identificativo



13. Sezione sicurezza

La sicurezza del sistema M2M è stata progettata seguendo le best practice dello sviluppo backend moderno.

Gestione variabili d'ambiente

Le informazioni sensibili (API key e URL del database) non vengono mai inserite direttamente nel codice sorgente, ma sono gestite tramite file:

`.env.local`

Vantaggi:

- protezione delle credenziali sensibili
- esclusione dal repository GitHub
- separazione tra codice e configurazione

Autenticazione utenti (Supabase Auth)

Il sistema utilizza Supabase Authentication per:

- login e logout utenti
- gestione sessioni
- identificazione dei ruoli

Gli utenti sono suddivisi in:

- sala
- cucina
- amministrazione

Controllo accessi (RBAC)

Il sistema implementa un controllo basato sui ruoli (Role-Based Access Control):

- la sala può creare ordini
- la cucina può aggiornare lo stato degli ordini
- l'amministratore ha controllo completo

Protezione API

Le API sono protette tramite:

- verifica della sessione utente
- controlli lato backend Node.js

- Supabase Row Level Security (RLS)

Risultato

Questa architettura di sicurezza garantisce:

- protezione dei dati sensibili
- accesso controllato alle funzionalità
- riduzione dei rischi di accessi non autorizzati

14. Possibili miglioramenti futuri

Il progetto M2M può essere ulteriormente ampliato con funzionalità avanzate.

Applicazione mobile

Sviluppo di una versione mobile per migliorare l'accessibilità di camerieri e cucina.

Notifiche push

Implementazione di notifiche in tempo reale per:

- nuovi ordini
- aggiornamenti di stato
- eventi urgenti

Dashboard analytics

Aggiunta di un pannello statistico per:

- numero ordini giornalieri
- piatti più richiesti
- tempi medi di preparazione

Integrazione AI

Possibili funzionalità intelligenti:

- previsione ordini frequenti
- ottimizzazione del menu
- analisi dei tempi di servizio

Gestione magazzino

Sistema per:

- monitoraggio ingredienti
- controllo scorte
- riordino automatico

Deploy completo online

Pubblicazione del progetto su cloud:

- frontend su Vercel
- backend e database su Supabase

Conclusione miglioramenti

Questi sviluppi permetterebbero di trasformare M2M da progetto didattico a **sistema gestionale completo di livello professionale e scalabile**.